Creazione Progetto

Installazione [node.js]
Verifica versione
- node -v
- npm -v

Installazione Angular CLI
nel terminale:
- npm install -g @angular/cli
- ng version

Se non fa Installare Angular CLI, è a causa dei permessi per l'installazione di pacchetti globali con npm, quindi cambiamo la directory globale di npm.

1. Crea una cartella per i pacchetti globali nella tua home:
   `mkdir ~/.npm-global`
2. Configura npm per usare quella cartella:
   `npm config set prefix '~/.npm-global'`
3. Aggiungi la cartella al PATH (modifica `~/.zshrc` o `~/.bashrc`):
   `export PATH=~/.npm-global/bin:$PATH`
4. Applica le modifiche:
   `source ~/.zshrc  # o source ~/.bashrc`
5. Ora puoi installare Angular CLI **senza sudo**:
   `npm install -g @angular/cli`

Creazione di un progetto
- ng new nome-progetto
- cd nome-progetto

configurazione esempio base:

```
➜  prova1 ng new nome-progetto
 Would you like to enable autocompletion? This will set up your terminal so
pressing TAB while typing Angular CLI commands will show possible options and
autocomplete arguments. (Enabling autocompletion will modify configuration files
 in your home directory.) Yes
Appended `source <(ng completion script)` to `/Users/albrimbytyqi/.zshrc`. Restart your terminal or run the following to
autocomplete `ng` commands:
   source <(ng completion script)

Would you like to share pseudonymous usage data about this project with the
Angular Team
at Google under Google's Privacy Policy at https://policies.google.com/privacy.
For more
details and how to change this setting, see https://angular.dev/cli/analytics.
  No
Global setting: disabled
Local setting: No local workspace configuration file.
Effective status: disabled
✔ Which stylesheet format would you like to use? CSS          [
https://developer.mozilla.org/docs/Web/CSS                    ]
✔ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation
(SSG/Prerendering)? No
✔ Do you want to create a 'zoneless' application without zone.js? No
✔ Which AI tools do you want to configure with Angular best practices?
https://angular.dev/ai/develop-with-ai None
```

Avvio server di sviluppo

- ng serve --open

Codice

Component:

Parti principali dei componenti:
- @Component: contiene configurazioni usate da Angular
- Template: controlla cosa caricare nel DOM
- CSS selector: definisce come viene usato il componente nel HTML
- Typescript class: gestione del comportamento

Creazione di un Componente:
- ng generate component **nome** --standalone

Reattività con Signal

Signal:

Use the signal function to create a signal for holding local state
- const firstName = signal('Morgan'); //Def
- console.log(firstName()); // Get
- firstName.set('Jaime'); // Set
- firstName.update(name => name.toUpperCase()) // Update

Computed expressions:

A computed is a signal that produces its value based on other signals.
- const firstNameCapitalized = computed(() => firstName().toUpperCase());

Definibile una singola volta, il valore si aggiorna in automatico, quando si modifica il valore di un signal.

Interfacce dinamiche con template:

Mostrare testo dinamico

- Usa **double curly braces** {{ }} per collegare i dati del componente al template.
- Se il valore cambia (ad esempio una **signal**), il template si aggiorna automaticamente.

Proprietà e attributi dinamici

- **Proprietà DOM**: usa **[property]** per legare valori dinamici.

  ```
  <button [disabled]="isValidUserId()">Save changes</button>
  ```
- **Attributi HTML**: usa **[attr.name]** per legare attributi che non hanno proprietà corrispondente:

  ```
  <ul [attr.role]="listRole()"></ul>
  ```

Angular aggiorna automaticamente **DOM e attributi** quando cambiano le signals.

Gestione eventi

- Usa **parentesi (event)** per ascoltare eventi e chiamare metodi del componente:

  ```
  <button (click)="cancelSubscription()">Cancel subscription</button>
  ```
- Se vuoi passare l'evento stesso:

  ```
  <button (click)="cancelSubscription($event)">Cancel subscription</button>
  cancelSubscription(event: Event) { … }
  ```

Controllo del flusso (@if / @for)

- Mostrare/nascondere parti del template:

  ```
  @if (isAdmin()) {
    …
  } @else {
    …
  }
  ```
- Ripetere template su array di dati:

  ```
  <ul>
    @for (badge of badges(); track badge.id) {
      <li>{{badge.name}}</li>
    }
  </ul>
  ```

track aiuta Angular a **collegare dati e DOM**, ottimizzando aggiornamenti.

Dependency Injection

Creazione Service

- ng generate service calculator

Cosa sono i Services

- I servizi sono pezzi di codice riutilizzabili che contengono logica da condividere tra componenti.
- Servono a centralizzare il codice e gestirlo da un unico punto di verità.
- Sono definiti con il decoratore @Injectable:

Come usare un servizio

- **Importa il servizio** nel componente.
- **Inietta il servizio** con la funzione inject() di Angular 20+.
- Usa i metodi del servizio nel componente.

Property Binding

In Angular, le parentesi quadre [] sono usate per il data binding delle proprietà.
tra le [] inseriamo una proprietà HTML e le assegnamo una variabile, Angular aggiorna automaticamente la proprietà.
Le parentesi quadre indicano che stai collegando una proprietà del DOM a una variabile o espressione del componente, non a una stringa fissa.

es: <img alt="photo" [src]="imageURL"> (imageURL è un signal)

Event Handling

Event handling abilita funzionalità interattive nelle app web. Offre agli sviluppatori la possibilità di rispondere alle azioni degli utenti, come la pressione di pulsanti, l'invio di moduli e altro ancora.
In Angular, l'associazione agli eventi avviene tramite la sintassi delle parentesi (). Su un dato elemento, si racchiude l'evento a cui si desidera associare tra parentesi e si imposta un gestore eventi.

es: <button (click)="greet()"> (greet() è un metodo)

Component input properties

Serve a passare dati da un componente padre a uno figlio. si definisci come una funzione **input()**. Restituisce un InputSignal, quindi per leggere il valore si usa property().

es: classe figlio: **occupation** = input<string>();
componente padre: <app-user **occupation**="Angular Developer"></app-user> + imports: [...]

Component output properties

Serve a mandare dati o notificare eventi dal figlio al genitore. Si definisce con **output<T>()**. Il genitore può ascoltare l'evento con la sintassi **(nomeEvento)="metodo($event)"**.

Figlio: nel template:

```
<button (click)="onClick()">Increment count</button>`,})
```

nella classe:

```
count = 0;
incrementCountEvent = output<number>();
onClick() {
  this.count++;
  this.incrementCountEvent.emit(this.count);
}
```

Padre: nel template:

```
<app-child (incrementCountEvent)="updateTotal($event)" />
```

nella classe:

```
total = 0;
  updateTotal(newValue: number) {
  this.total = newValue;
}
```

Deferrable View

Permette di rinviare il caricamento di un componente fino a quando (serve a migliorare le performance) :

- il browser è inattivo,
- il componente entra in viewpoint
- viene eseguito un evento, ecc

```
@defer {
  <comments />
} @placeholder {
  <p>Future comments</p>
} @loading {
  <p>Loading comments...</p>
}
```

<comments /> verrà caricato solo quando il browser è **idle**.
// Placeholder (contenuto statico mostrato prima)
// Loading (mostrato mentre il componente si sta caricando)

```
@defer (on viewport) {
  <comments />
} @loading (minimum 2s) {
  <p>Loading comments...</p>
}
```

Trigger su viewport: carica il componente solo quando entra nello schermo (ottimo per contenuti in basso)

// Minimum duration (evita flickering se il caricamento è molto rapido)

Optimizing images (NgOptimizedImage)

Per usare la direttiva vedo prima importarla:

- **imports**: [NgOptimizedImage]

Sostituzione di src con **ngSrc**.
Attributi richiesti:

- **Obbligatorio** specificare **width** e **height** per evitare layout shift.
- se non si riesce ad usare valori fissi, usare **fill** (il container deve avere position: relative|absolute|fixed).

Si possono marcare le immagini importanti con : **property**.
Tutte le immagini senza property sono caricate in **lazy loading**.
Per usare una base URL personalizzata, Image Loader:

```
import { provideImgixLoader } from '@angular/common';

@Component({
  ...
  providers: [
    provideImgixLoader('https://my.base.url/')
  ]
})
export class ImagesComponent {}
…
<img ngSrc="image.png" width="600" height="400" />
```

URL finale → https://my.base.url/image.png

Routing Overview

Basato dal file app.routes.ts

```
// app.routes.ts conf base
import { Routes } from '@angular/router';
```

```
export const routes: Routes = [];
```

Un array di oggetti, ognuno con una rotta (path/componente/…).

Configuarzione app.config.ts

```
// app.config.ts
 import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes)],
};
```

Poi nel codice interessato:

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  template: `
    <nav>
      <a href="/">Home</a> |
      <a href="/user">User</a>
    </nav>

    <!-- Qui Angular caricherà i componenti delle rotte -->
```

```
  <router-outlet />
  `,
})
export class App {}
```

Define a Route

1. Ogni **rotta è un oggetto dentro l'array routes**.
2. Una rotta ha almeno:
   - **path**: l'URL (es. '' → /, 'user' → /user).
   - **component**: il componente da mostrare dentro <router-outlet>.
3. Puoi anche aggiungere proprietà extra come title (titolo della pagina).

Esempio route:

```
import { Routes } from '@angular/router';
import { Home } from './home/home';
import { User } from './user/user';

export const routes: Routes = [
  {
    path: '',
    title: 'App Home Page',
    component: Home,
  },
  {
    path: 'user',
    title: 'User Profile',
    component: User,
  },
];
```

Usare RouterLink per la Navigazione

- Quando usi il tag **<a href="/user">User</a>** il browser ricarica tutta la pagina (HTML, CSS, JS, dati, ecc..)
- **<a routerLink="/user">User</a>** Angular intercetta il click e carica solo il componente associato alla rotta, non l'intera pagina.

Forms

Per usarlo serve importare **FormsModule**

Esempio form non funzionante:

```
<label for="framework">
  Favorite Framework:
  <input id="framework" type="text" />
</label>
```

Esempio Form funzionante:

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-user',
  standalone: true,
  imports: [FormsModule], // <-- aggiunto qui
  template: `
    <label for="framework">
      Favorite Framework:
      <input id="framework" type="text" [(ngModel)]="favoriteFramework" />
    </label>

    <p>Hai scritto: {{ favoriteFramework }}</p>
```

```
  `,
})
export class User {
  favoriteFramework = '';  // proprietà collegata all'input
}
```

- Con [(ngModel)]="favoriteFramework" diciamo:
  - **[ ] property binding** → il valore di favoriteFramework dal componente → input
  - **( ) event binding** → quando l'utente scrive, l'input aggiorna favoriteFramework
- Questo è chiamato **two-way data binding** (banana in a box 🍌📦).

Recuperare il valore nei Forms

Esempio:

```
@Component({
  selector: 'app-user',
  template: `
    <label for="framework">
      Favorite Framework:
      <input id="framework" type="text" [(ngModel)]="favoriteFramework" />
    </label>

    <button (click)="showFramework()">Show Framework</button>
  `,
})
export class User {
  favoriteFramework = '';

  showFramework() {
    alert(this.favoriteFramework); // legge il valore scritto nell'input
  }
```

```
}
```

Reactive Forms

Permettono di gestire i form in maniera **programmatica e reattiva** invece che solo nel **template**

- **Template-driven forms** → la logica è scritta soprattutto **nell'HTML** (con ngModel e simili).
- **Reactive forms** → la logica è scritta nel **TypeScript**: tu crei un **FormGroup** e **FormControl**, e il **template** è collegato a questi oggetti.

Esempio codice:

```
import { Component } from '@angular/core';
import { ReactiveFormsModule, FormControl, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-root',
  template: `
    <h2>Profile Form</h2>

    <form [formGroup]="profileForm" (ngSubmit)="handleSubmit()">
      <label>
        Name
        <input type="text" formControlName="name" />
      </label>

      <label>
        Email
        <input type="email" formControlName="email" />
      </label>
```

```
    <button type="submit">Submit</button>
  </form>

  <p>Name: {{ profileForm.value.name }}</p>
  <p>Email: {{ profileForm.value.email }}</p>
`,
imports: [ReactiveFormsModule],
standalone: true
})

export class App {
  // 2. Crea il FormGroup con i suoi FormControl
  profileForm = new FormGroup({
    name: new FormControl(''),   // valore iniziale vuoto
    email: new FormControl(''),  // valore iniziale vuoto
  });

  // 3. Metodo per gestire il submit
  handleSubmit() {
    alert(
      this.profileForm.value.name + ' | ' + this.profileForm.value.email
    );
  }
}
```

Vantaggi dei Reactive Forms

- Controlli completi da TypeScript (puoi aggiungere/aggiornare campi dinamicamente).
- Validazioni potenti (sincronizzate e asincrone).
- Facile testabilità.
- Più adatti a form complessi e grandi.

Validazione dei Forms

Con i Reactive Forms la **validazione si fa lato TypeScript**, agganciandola direttamente ai FormControl.

Importazione di Validators.
Angular ti fornisce delle validazioni già pronte (required, email, minLength, maxLength, ecc.).

```
import { Validators } from '@angular/forms';

…

export class App {
  profileForm = new FormGroup({
    name: new FormControl('', Validators.required),  // obbligatorio
    email: new FormControl('', [Validators.required, Validators.email]), // obbligatorio e
formato email valido
  });

  handleSubmit() {
    alert(this.profileForm.value.name + ' | ' + this.profileForm.value.email);
  }
}
```

Possiamo verificare la validazione del form al bottone del form.

```
<form [formGroup]="profileForm" (ngSubmit)="handleSubmit()">
  <label>
    Name
    <input type="text" formControlName="name" />
  </label>

  <label>
    Email
```

```
    <input type="email" formControlName="email" />
  </label>

  <!-- Disabilitato se il form non è valido -->
  <button type="submit" [disabled]="!profileForm.valid">Submit</button>
</form>

<p>Name: {{ profileForm.value.name }}</p>
<p>Email: {{ profileForm.value.email }}</p>
```

- profileForm.valid → diventa true solo se tutti i campi rispettano i loro validators.

Creating an injectable service

Un service è semplicemente una classe che contiene logica riutilizzabile (ad esempio: gestire i dati, chiamare API, manipolare stati).
Per far sì che Angular lo riconosca e lo possa iniettare dove serve, bisogna decorarlo con @Injectable.

1. **Creare il file del service**
   Di solito si usa la convenzione: nome.service.ts.
   Nel tuo caso: car.service.ts

2. Importare Injectable e decorare la classe

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root', // disponibile in tutta l'app
})
```

```
export class CarService {
  // Qui scriverai la logica (esempio: gestione auto)
  getCars() {
    return ['Tesla', 'BMW', 'Audi'];
  }
}
```

Inject-based dependency injection

Usare il service dentro un componente grazie alla dependency injection.

Iniettarlo in un componente con inject():

```
import { Component, inject } from '@angular/core';
import { CarService } from './car.service';

@Component({
  selector: 'app-root',
  template: `<p>Car Listing: {{ display }}</p>`,
})
export class App {
  // Inject del service
  carService = inject(CarService);

  // Usiamo il service per generare una stringa
  display = this.carService.getCars().join(' ⭐ ');
}
```

Pipes

Importa la **LowerCasePipe**.

Le pipes sono **funzioni di trasformazione** che puoi usare **direttamente nell'HTML** del template.
Servono per **trasformare i dati solo per la visualizzazione**, senza modificare il valore originale nella classe.

Esempio:

```
@Component({
  selector: 'app-root',
  template: `{{ username | lowercase }}`,
  imports: [LowerCasePipe], // 👈 Import della pipe
})
export class App {
  username = 'YoungTECH';
}
```

```
youngtech
```

Le pipes prendono un valore e lo trasformano solo nella view (il valore originale in username resta YoungTECH).

Formattazione dei dati con pipes

Quando vuoi configurare il comportamento di una pipe, usi la sintassi:

```
{{ valore | pipeName:parametro }}
```

Se la pipe accetta più parametri:

```
{{ valore | pipeName:param1:param2 }}
```

Esempio:

```
@Component({
  selector: 'app-root',
  template: `
```

```
    <li>Number with "decimal" {{ num | number:"3.2-2" }}</li>
  `
})
export class App {
  num = 123.4567;
}
```

- "3.2-2" significa:
  - 3 → minimo 3 cifre intere
  - 2 → minimo 2 cifre decimali
  - -2 → massimo 2 cifre decimali
    👉 Output: 123.46

Creare una pipe personalizzata

Una **custom pipe** ti permette di trasformare i dati come vuoi tu, non solo con le pipes integrate.

Passi per creare la ReversePipe

1. Creare il file della pipe (reverse.pipe.ts)

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'reverse'
})

export class ReversePipe implements PipeTransform {
  transform(value: string): string {
    let reverse = '';
    for (let i = value.length - 1; i >= 0; i--) {
      reverse += value[i];
```

```
    }
    return reverse;
  }
}
```

- **@Pipe({ name: 'reverse' })** → il nome che userai nel template (| reverse)
- **transform()** → è il cuore della pipe, contiene la logica di trasformazione

2. Usare la pipe in un componente (app.ts)

```
import { Component } from '@angular/core';
import { ReversePipe } from './reverse.pipe';

@Component({
  selector: 'app-root',
  template: `
    <p>Reverse Machine: {{ word | reverse }}</p>
  `,
  imports: [ReversePipe]
})
export class App {
  word = 'Angular';
}
```